

International Islamic University Islamabad

Faculty of Engineering and Technology

Department of Electrical and Computer Engineering



AI & Machine Learning Lab

Experiment No. 4: Raspberry Pi Setup for AI and Machine Learning

Name of Student: ..[Bushra Nazir](#).....

Registration No.: ...[1071-F22](#).....

Date of Experiment: ...[27-2-2026](#).....

Submitted To: ...[Engr.Asad](#).....

Raspberry Pi Setup for AI and Machine Learning

Objectives

- ✓ To understand the hardware and software setup of the Raspberry Pi for development purposes.
- ✓ To explore the Raspberry Pi OS environment and utilize Python for basic programming tasks.
- ✓ To interface and control GPIO pins for foundational input/output operations relevant to AI/ML projects.

Equipment Required

- Raspberry Pi board (Model 3B/4B recommended)
- MicroSD card (16GB or higher, Class 10)
- Power supply for Raspberry Pi
- HDMI cable and monitor (or laptop with VNC/SSH setup)
- USB keyboard and mouse
- Breadboard
- LEDs, resistors, push buttons
- Jumper wires (male-to-male, male-to-female)
- Internet connectivity (Wi-Fi or Ethernet)

Introduction to Raspberry Pi

The Raspberry Pi is a small, affordable, single-board computer developed by the Raspberry Pi Foundation in the United Kingdom. It was first launched in 2012 with the goal of promoting computer science education in schools and developing countries. Over the years, it has evolved into a powerful platform used in a wide range of applications, including IoT, robotics, home automation, and AI/ML projects.

Raspberry Pi serves as a bridge between general-purpose computing and embedded system development. Its scope extends from basic programming education to advanced applications such as:

- Machine learning inference and edge computing

- Data collection and real-time analytics
- Smart home and automation systems
- Computer vision and image processing
- Prototyping intelligent devices and IoT solutions

With its Linux-based environment and support for Python and other high-level languages, Raspberry Pi provides a versatile ecosystem for development.

Difference Between Raspberry Pi and Microcontrollers (e.g., Arduino)

Raspberry Pi is a full-fledged single-board computer with an operating system, capable of running complex applications, multitasking, and AI/ML programs. In contrast, microcontrollers like Arduino are limited-resource devices designed for simple, real-time control tasks without an OS.

Feature	Raspberry Pi	Arduino
Type	Single-board computer	Microcontroller board
Operating System	Runs full Linux-based OS (e.g., Raspbian)	No OS, runs firmware
Programming Language	Python, C/C++, Java, etc.	Mostly C/C++ via Arduino IDE
Processing Power	High (Quad-core CPU, 1GB+ RAM)	Low (e.g., 8-bit AVR with 2KB SRAM)
Applications	Multi-tasking, AI/ML, media, networking	Real-time control, simple automation
Connectivity	HDMI, USB, Ethernet, Wi-Fi, Bluetooth	Basic USB/serial

Raspberry Pi is ideal when an operating system, file system, or high processing power is required, especially in AI/ML applications. Arduino is best suited for real-time control tasks like sensor reading and motor control.

Types of Raspberry Pi Boards

The Raspberry Pi family includes several models:

- **Raspberry Pi 1, 2, 3, and 4:** Standard models with increasing performance
- **Raspberry Pi Zero / Zero W:** Ultra-compact and low-power
- **Raspberry Pi 400:** Integrated into a keyboard for ease of use
- **Raspberry Pi Compute Module:** Designed for industrial and embedded applications

Each version brings improvements in speed, RAM, connectivity, and peripherals support.

Similar Boards

Other single-board computers that offer comparable features include:

- **NVIDIA Jetson Nano** – Designed for AI/ML with GPU acceleration
- **BeagleBone Black** – Strong in I/O and real-time performance
- **Banana Pi, Orange Pi, and Odroid** – Raspberry Pi alternatives with varying specs and costs

Why Raspberry Pi is a Better Choice for AI and ML

- ✓ Cost-effective and widely available
- ✓ Runs full Linux OS with support for AI/ML frameworks like TensorFlow Lite, OpenCV, and PyTorch
- ✓ Good processing power and RAM for edge inference tasks
- ✓ Active community and extensive online resources
- ✓ GPIO access for real-world data collection and automation

Its combination of affordability, flexibility, and performance makes Raspberry Pi an excellent platform for introducing students to AI and Machine Learning in real-world embedded applications.

Raspberry Pi Architecture

The Raspberry Pi is built around the System on Chip (SoC) concept, where the processor, memory, and I/O peripherals are integrated into a single chip. It combines the capabilities of a general-purpose computer with input/output features typically found in embedded systems,

making it a hybrid platform suited for a variety of applications, including AI and ML.

Core Components of Raspberry Pi Architecture

- **Processor (CPU):**

Raspberry Pi uses ARM-based CPUs, commonly from the Broadcom BCM series. For example, Raspberry Pi 4 uses a 1.5 GHz Quad-Core ARM Cortex-A72 (64-bit) processor. This architecture is power-efficient and capable of multitasking, suitable for running AI models and Python-based ML scripts.

- **RAM (Memory):**

Depending on the model, it ranges from 512MB to 8GB LPDDR4, allowing the handling of memory-intensive applications like computer vision and data processing.

- **GPU (Graphics Processing Unit):**

Integrated VideoCore IV GPU enables HDMI video output and basic graphical processing. For AI tasks involving image data, it supports basic visualization and rendering.

- **Storage:**

Raspberry Pi does not come with built-in storage; instead, it uses a microSD card as the primary boot and storage medium. Some models also support USB booting.

- **Power Supply:**

Typically requires a 5V/2.5A to 5V/3A power adapter, depending on the model and connected peripherals.

Interfaces and Connectivity

Raspberry Pi supports a widerange of interfaces that make it flexible for hardware interfacing and data communication:

- **GPIO (General Purpose Input/Output)** – 40-pin header with digital I/O, PWM, I2C, SPI, and UART support

- **HDMI Port** – Connectstomonitorsand TVs
- **USB Ports** –USB2.0/3.0forperipheralslikekeyboard, mouse, and storage
- **Ethernet Port** –Wirednetworkconnectivity
- **Wi-Fi and Bluetooth** –Built-inwirelesscommunication on most models
- **Camera Serial Interface (CSI)** –ForconnectingRaspberry Pi Camera Module
- **Display Serial Interface (DSI)** –ForofficialRaspberry Pi touchscreens
- **Audio Jack** – For analog audio output
- **MicroSD Slot** – For OS and file storage

Specifications Table (Example: Raspberry Pi 4 Model B)

Component	Specification
CPU	Quad-core Cortex-A72 (ARM v8, 64-bit, 1.5 GHz)
RAM Options	2GB, 4GB, 8GB LPDDR4
GPU	Broadcom VideoCore VI
Storage	microSD, USB boot support
USB Ports	2 × USB 3.0, 2 × USB 2.0
GPIO	40-pin header (3.3V logic, various protocols)
Video Output	2 × micro HDMI (up to 4K)
Network	Gigabit Ethernet, Wi-Fi 802.11ac, Bluetooth 5
Camera Interface	1 × CSI port
Display Interface	1 × DSI port
Power Supply	5V/3A USB-C

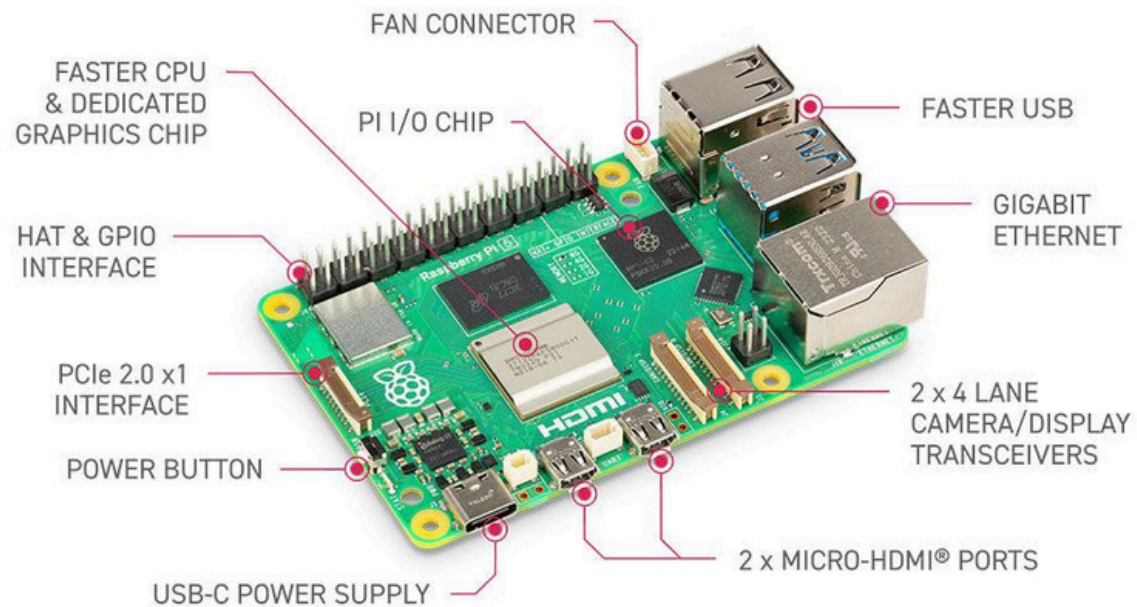


Figure: Raspberry Pi Interfaces

The General Purpose Input/Output (GPIO) pins on the Raspberry Pi are one of its most versatile and essential features, allowing the board to interface directly with various electronic components and hardware. These pins enable the Raspberry Pi to send digital signals to control devices like LEDs, buzzers, and relays, or to receive input from buttons, sensors, and other peripherals. Modern Raspberry Pi boards such as the Raspberry Pi 3 and 4 come with a 40-pin GPIO header, out of which 26 pins are configurable as general-purpose input or output. The remaining pins are assigned to power (3.3V, 5V) and ground connections.

GPIO pins operate at a logic level of 3.3 volts, which means caution must be taken to avoid connecting them directly to higher voltages, as this could damage the board. In addition to basic input/output operations, many of these pins support hardware communication protocols like I2C, SPI, and UART. These interfaces allow the Raspberry Pi to communicate efficiently with various digital sensors, displays, and other external devices, which are crucial for AI and ML projects involving real-world data.

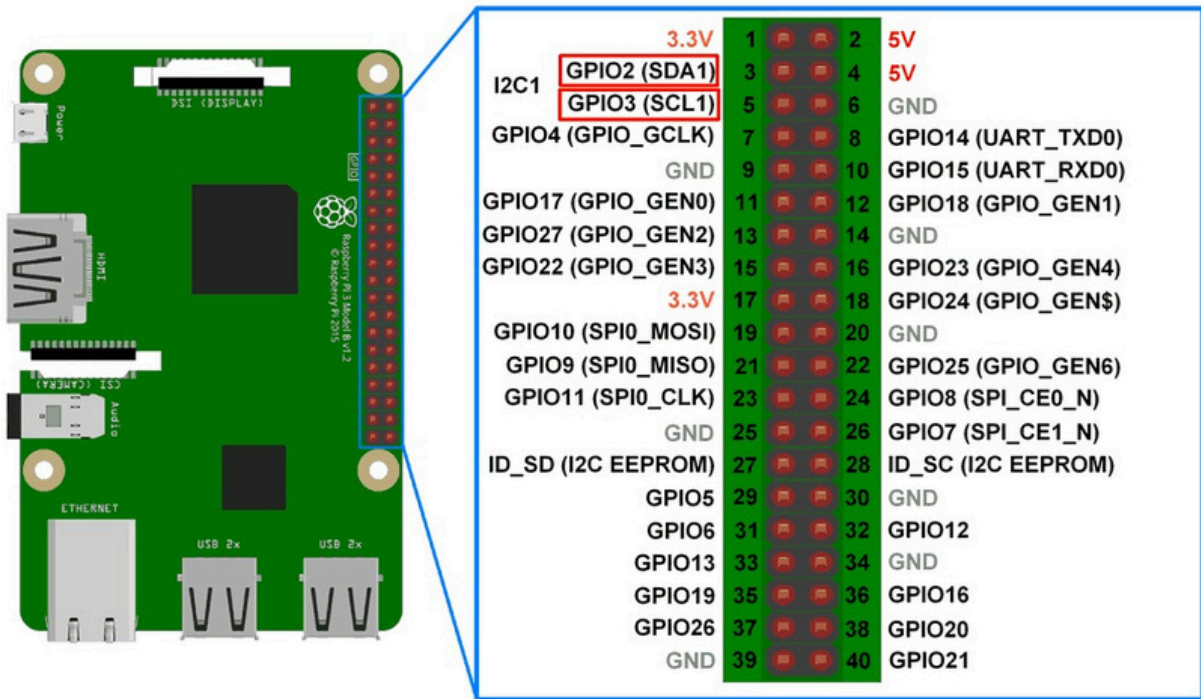


Figure: Raspberry Pi GPIOs

Setting Up Raspberry Pi for AI/ML Development

The setup of a Raspberry Pi is an essential first step before diving into AI and ML applications. This section will guide you through the process of getting your Raspberry Pi up and running, preparing it for programming and hardware interfacing.

Setup the pi board in the following steps.

Software setup

1. Download Raspberry Pi imager software from the link <https://www.raspberrypi.com/software/> according to your laptop/pc OS.
2. Install the Imager software and open it.

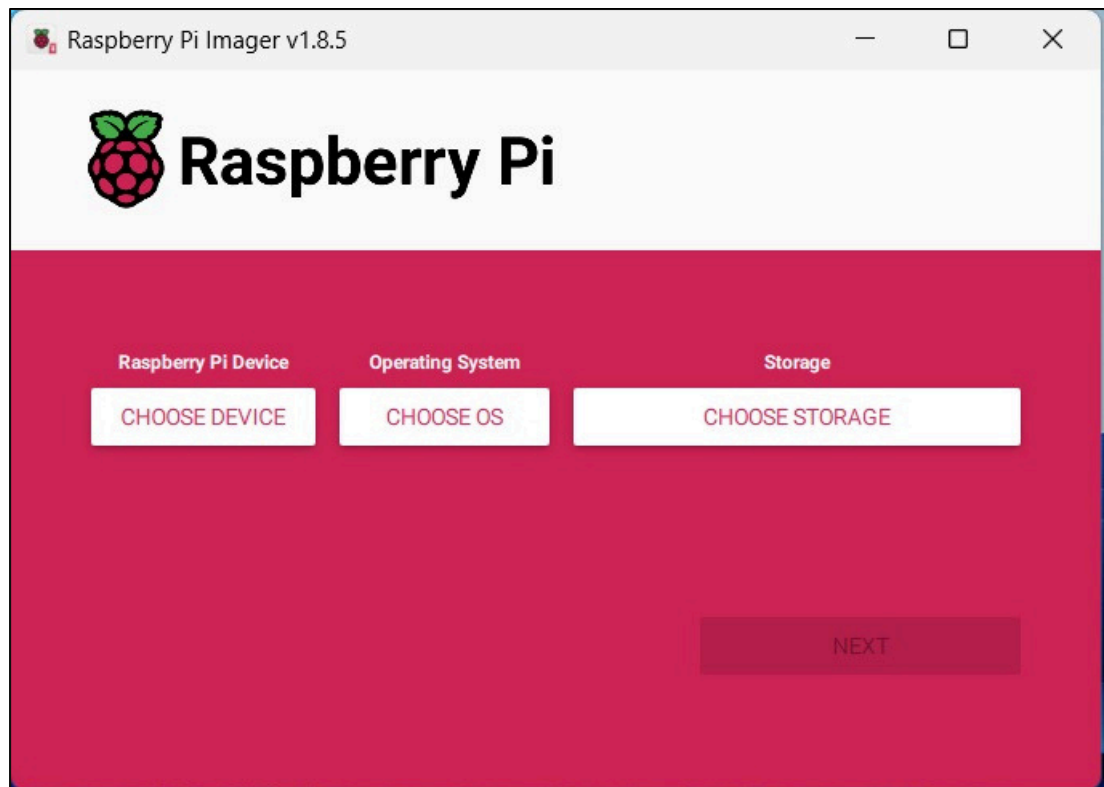


Figure: Raspberry Pi Imager

3. Insert micro SD card into card reader and plugin the card reader into laptop/PC.

4. Now from imager software

- Click on Choose Device and select Raspberry Pi 4 (or whichever you have)
- Click on Choose OS and select Raspberry PI OS (64 bit)
- Click on Choose Storage and select the memory card drive
- Click Next
- In the Next window, it will ask about Edit Settings, select NO.
- In the next window, it will ask about permission to overwrite the contents of the memory card by erasing the previous data, if any. Select Yes.
- Click Next and it will start downloading and install the OS in memory card. Wait till it writes and verifies 100%.
- Once it is done, remove SD card from card reader.

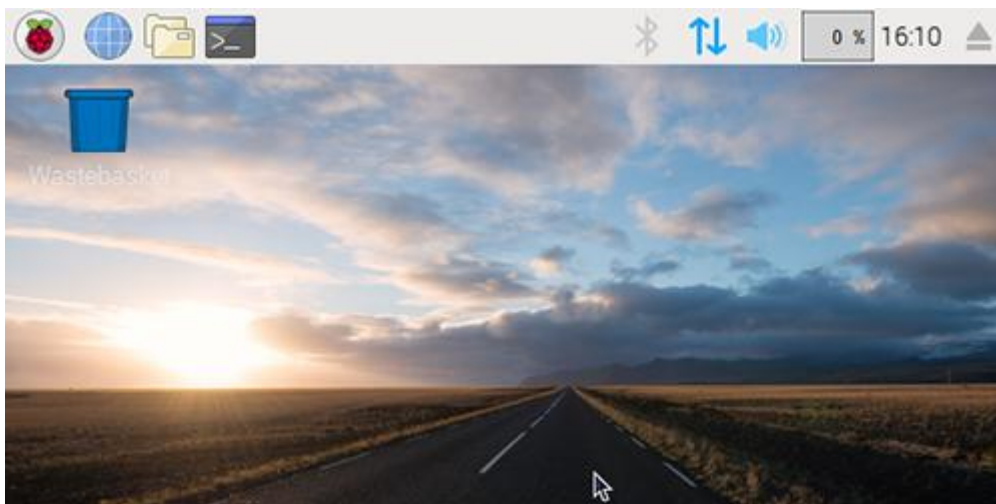
Hardware setup

5. Take a Raspberry Pi board, a power supply (5v/3A for Raspberry Pi 4), an HDMI cable (HDMI to Micro HDMI cable), a keyboard, a mouse, and a monitor.
6. Connect the HDMI cable to a monitor, plug in the USB keyboard and mouse, and ensure an internet connection via Ethernet or Wi-Fi.
7. insert the microSD card into the Raspberry Pi and power it on.

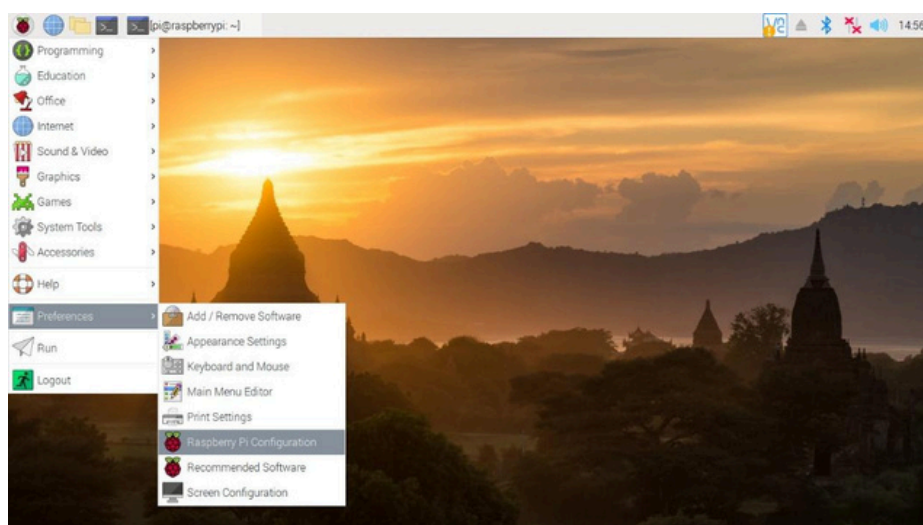


Initial Boot:

- Power on the Raspberry Pi by connecting the power supply. The system will boot into the Raspberry Pi OS.
- You'll see the Raspberry Pi logo followed by a setup wizard.
- Complete the setup wizard by selecting your language, timezone, and connecting to a Wi-Fi network (Make sure you have same wifi connected to your laptop and Pi board).
- Once all set, you will see the Raspberry Pi desktop.



Raspberry pi desktop is similar to normal pc desktop except that it has Linux environment. Taskbar is on the above side with Pi menu (called start menu). Navigate along the taskbar menu and explore different options for a better understanding.



Console Mode

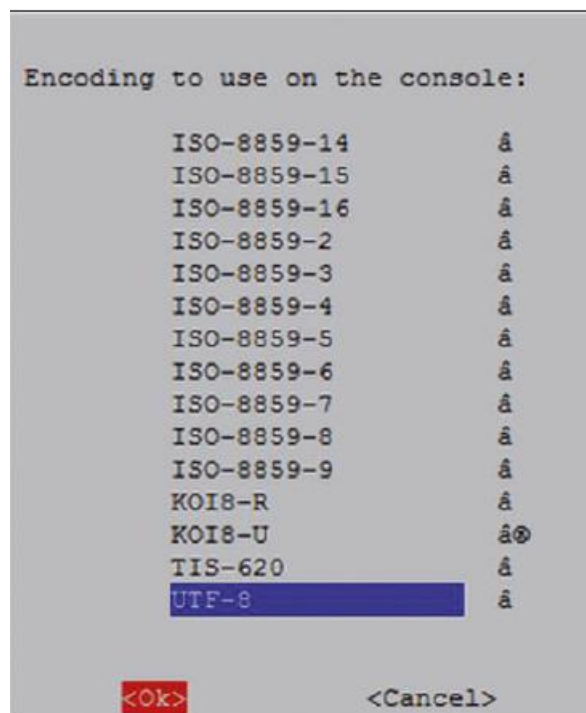
Console mode is a text-only interface that allows users to interact with the Raspberry Pi without the graphical desktop environment. It operates directly on virtual terminals (e.g., /dev/tty1) and is ideal for low-resource or headless setups. Unlike the terminal, which runs inside the desktop as an application, console mode provides direct access to the system shell at the hardware level. You can press Ctrl+Alt+F1 at any time to change to the Console mode. (To exit the mode Ctrl+Alt+F7)

Larger font in Console mode

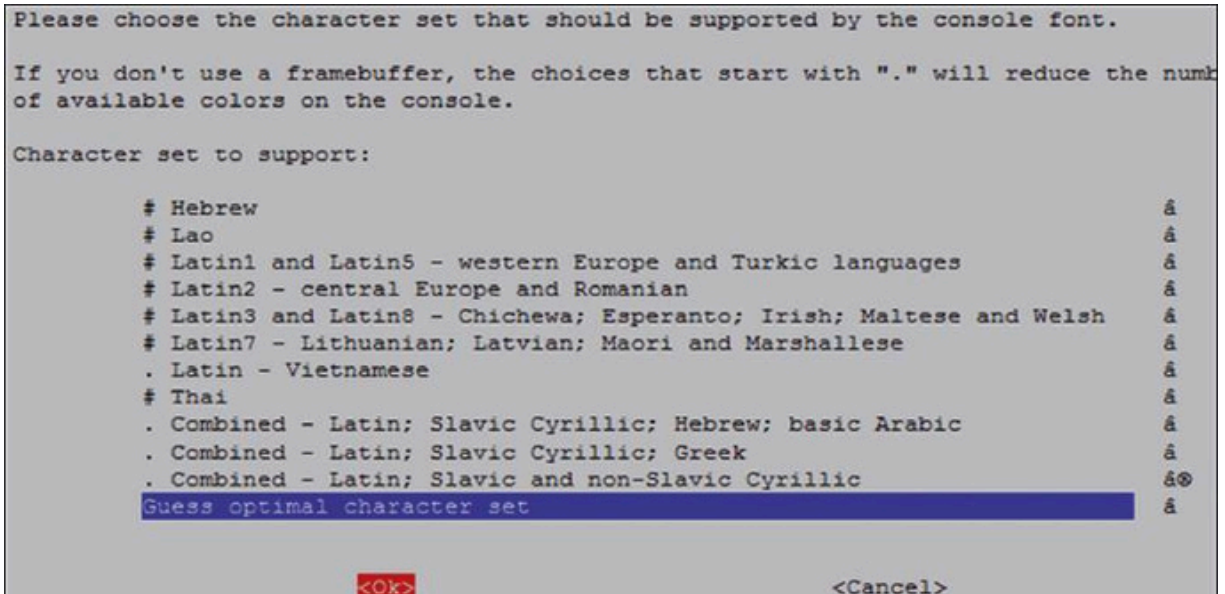
- Make sure you are in the Console mode
- Enter the following command:
-

```
pi@raspberrypi:~$ sudo dpkg-reconfigure console-setup
```

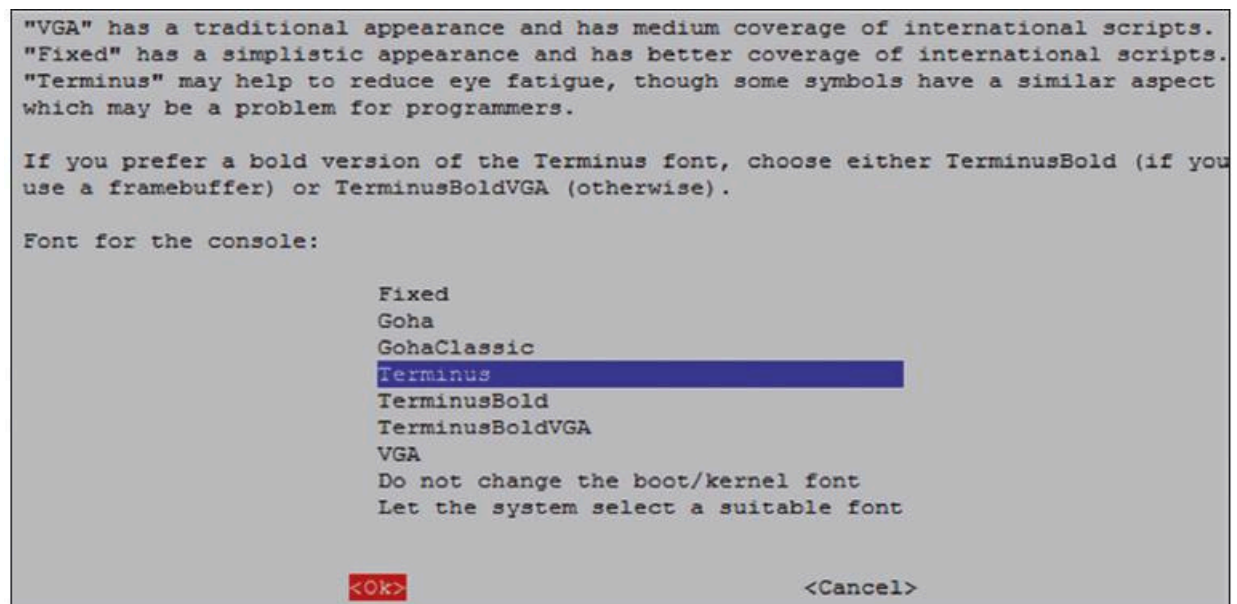
- **Select UTF-8 in the Package Configuration screen**



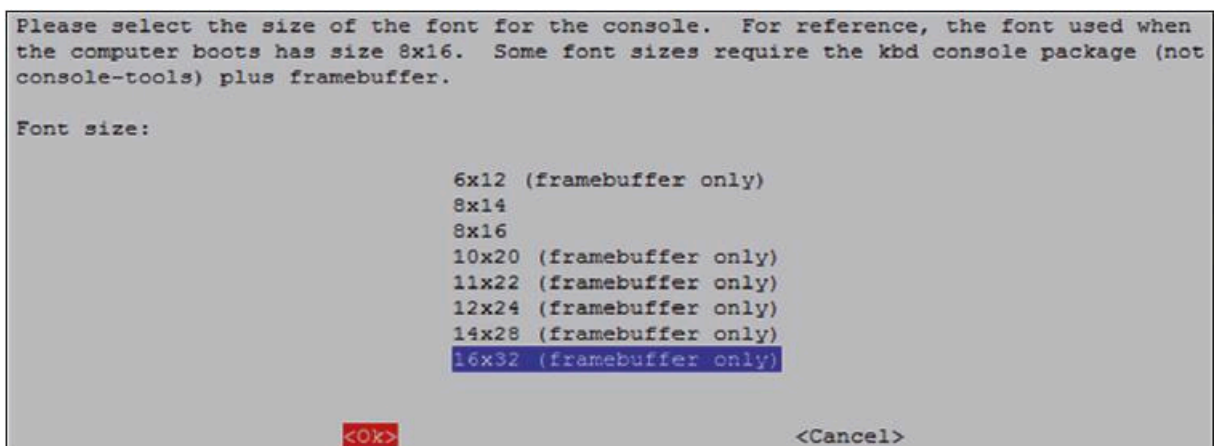
- Select Guess optimal character set



- Select Terminus



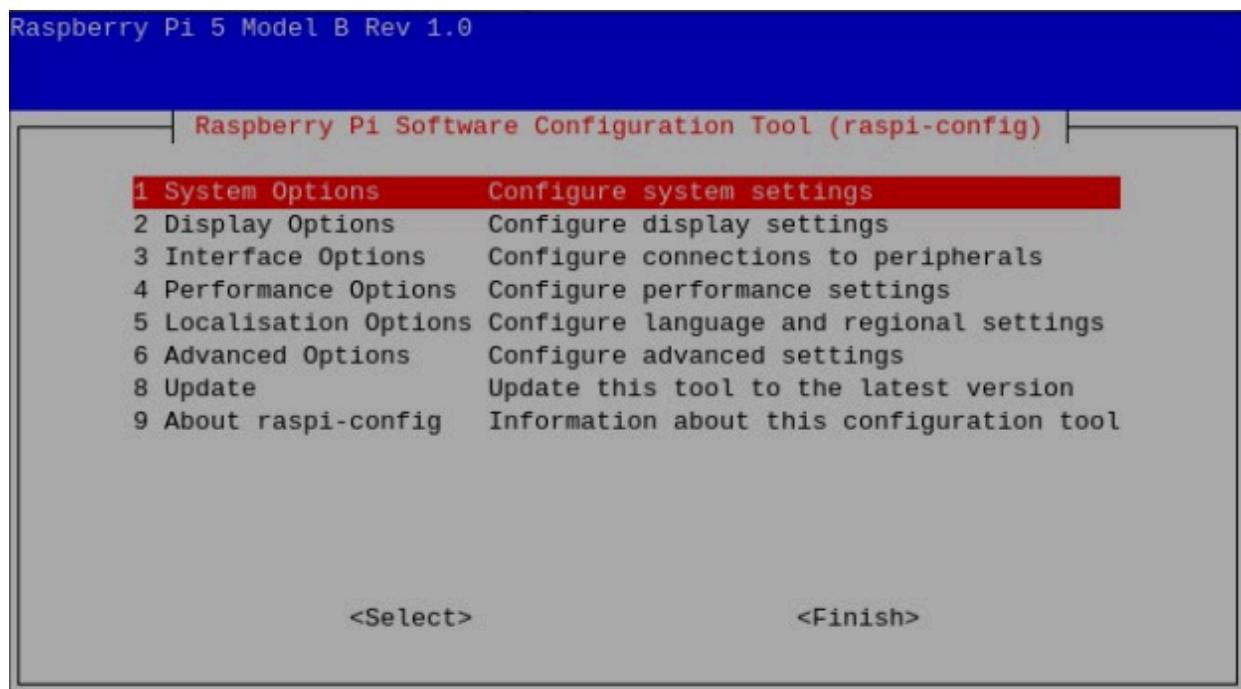
- Select font 16x32



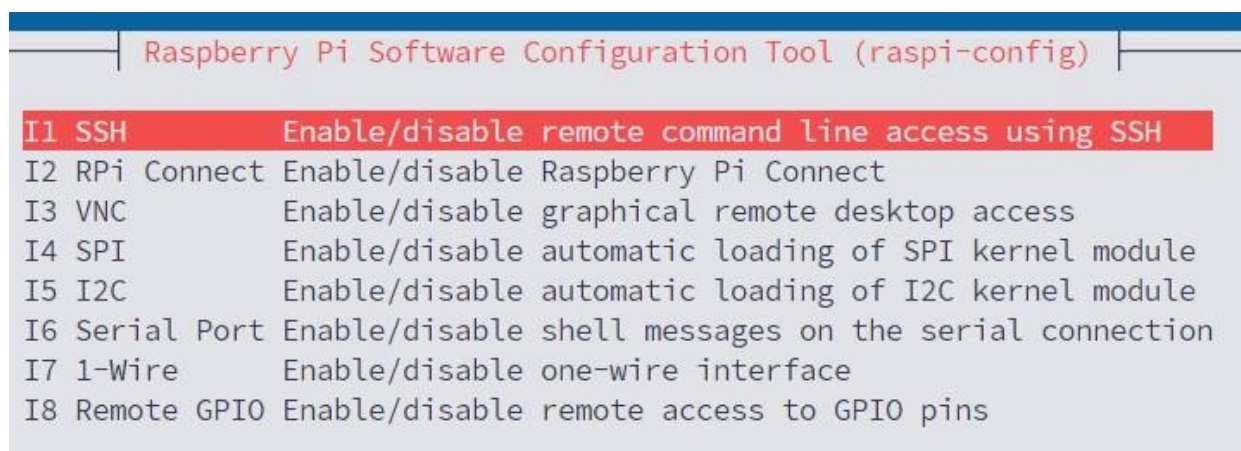
Accessing your Raspberry Pi 5 Console from your PC – the Putty program

In many applications, you may want to access your Raspberry Pi 5 from your PC. This requires enabling the SSH on your Raspberry Pi and then using a terminal emulation software on your PC. The steps to enable the SSH are as follows:

- Make sure you are in Console mode
- Type: **sudo raspi-config**
- Move down to **Interface Options**



- **Highlight SSH and press Enter**



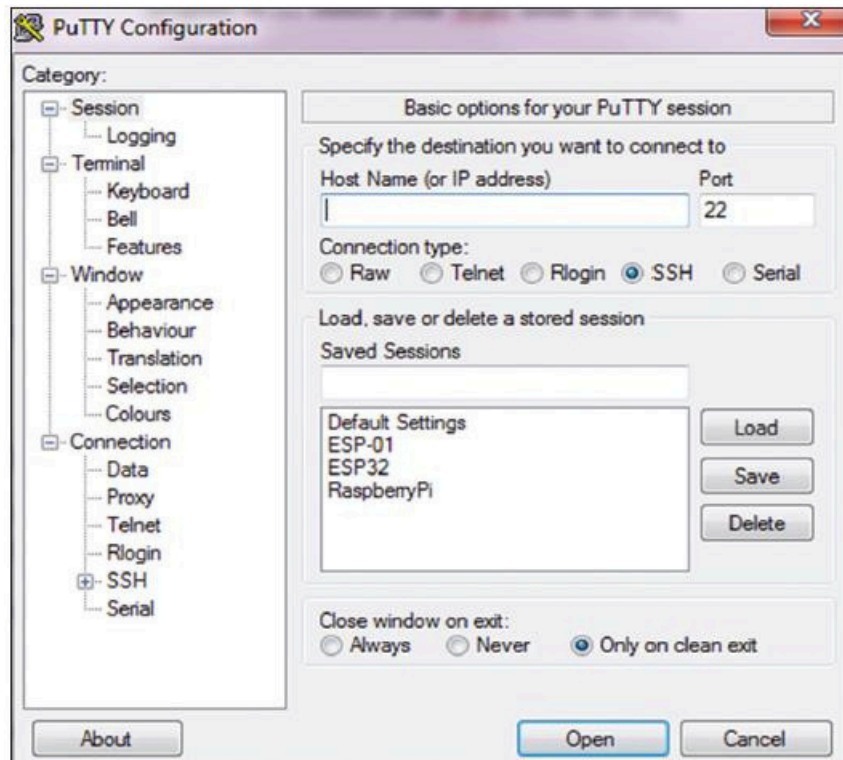
- Click **Yes** to enable SSH
- Click **OK**

- Move down and click Finish

You will now have to install a terminal emulation software on your PC. Download Putty from the following website

: <https://www.putty.org>

- Putty is a standalone program, and there is no need to install it. Simply double-click to run it. You should see the Putty startup screen as in Figure.



- Make sure that the Connection type is SSH and enter the IP address of your Raspberry Pi . You can obtain the IP address by entering the command **ifconfig** in console mode In this example, the IP address was: **192.168.1.251** (see under **wlan0**.)

```

pi@raspberrypi:~ $ ifconfig
eth0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    ether d8:3a:dd:77:b2:e2 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 107

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 105 bytes 9175 (8.9 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 105 bytes 9175 (8.9 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.251 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 2a00:23c7:868d:7b01:1562:5802:73c0:1ff6 prefixlen 64 scopeid 0x
<global>
    inet6 fe80::966a:a2d8:912:fa6b prefixlen 64 scopeid 0x20<link>

```

- Click **Open** in Putty after entering the IP address and selecting **SSH**
- The first time you run Putty, you may get a security message. Click **Yes** to accept this security alert.
- You will then be prompted to enter the Raspberry Pi username and password (you have setup in the very initial windows).

You can now enter all Console-based commands through your PC.

- To change your password, enter the following command:

```
pi@raspberrypi: ~ $ passwd
```

- To restart the Raspberry Pi, enter the following command:

```
pi@raspberrypi: ~ $ sudo reboot
```

• To shut down the Raspberry Pi, enter the following command. Never shut down by pulling the power cable, as this may result in the corruption or loss of files:

```
pi@raspberrypi: ~ $ sudo shutdown -h now
```

Accessing the Desktop GUI from your PC

If you are using your Raspberry Pi with a local keyboard, mouse, and display, you can skip this section. If, on the other hand, you want to access your Desktop remotely over the network, you will find that SSH services cannot be used. The easiest and simplest way to access your Desktop remotely from a computer is by using the VNC (Virtual

Network Connection) client and server. The VNC server runs on your Pi, and the VNC client runs on your computer.

It is recommended to use the **tightvncserver** on your Raspberry Pi 5. The steps are:

- Enter the following command in console mode

```
pi$raspberrypi:~ $ sudo apt-get install tightvncserver
```

- Run the tightvncserver:

```
pi$raspberrypi:~ $ tightvncserver
```

You will be prompted to create a password for remotely accessing the Raspberry Pi desktop. You can also set up an optional read-only password. The password should be entered every time you want to access the Desktop. Enter a password and remember your password.

- Start the VNC server after reboot by the following command:

```
pi$raspberrypi:~ $ vncserver :1
```

- You can optionally specify screen pixel size and colour depth in bits as follows:

```
pi$raspberrypi:~ $ vncserver :1 –geometry 1920x1080 –depth 24
```

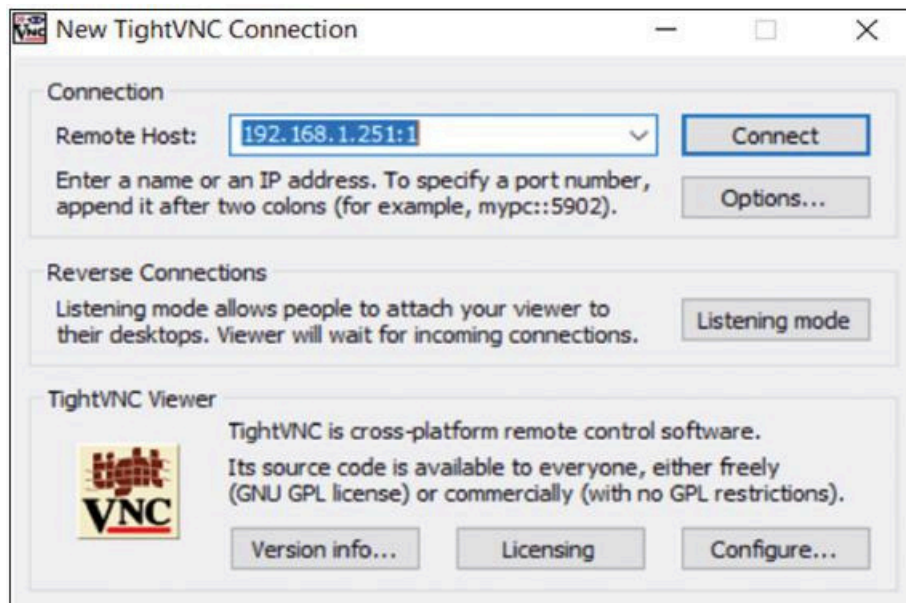
We must now set up a VNC viewer on our laptop (or desktop) PC. There are many VNC clients available, but the recommended one which is compatible with **TightVNC** is **TightVNC** for the PC, which can be downloaded from the following link:

<https://www.tightvnc.com/download.php>

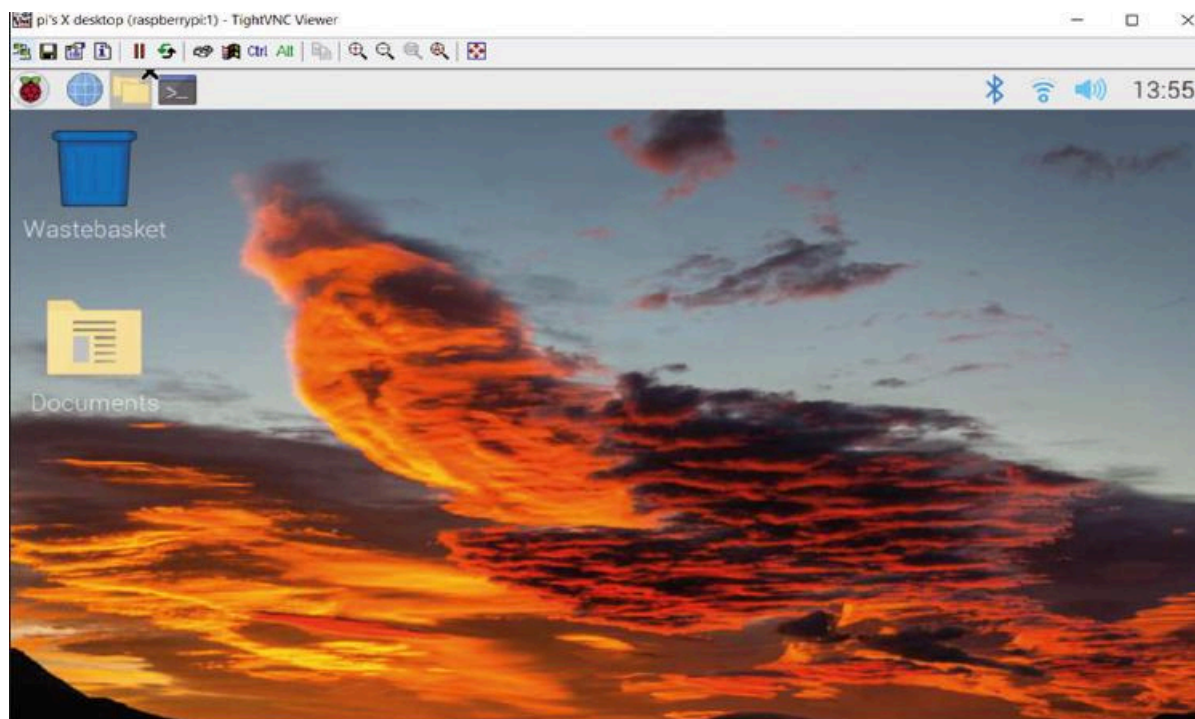
- Download and install the **TightVNC** software for your PC. You will have to choose a password during the installation.
- Start the **TightVNC Viewer** on your PC and enter the Raspberry Pi IP address

followed by **':1'**.

- Click Connect to connect to your Raspberry Pi



- Enter the password you have chosen earlier. You should now see the Raspberry Pi Desktop displayed on your PC screen



The VNC server is now running on your Raspberry Pi 5 and you have access to the Desktop GUI.

Assigning a static IP address to your Raspberry Pi 5

When you try to access your Raspberry Pi remotely over your local network, it is possible

that the IP address given by your Wi-Fi router can change from time to time. This is annoying as you have to find out the new IP address allocated to your Raspberry Pi. Without knowledge of the IP address, you cannot log in using SSH or VNC. In this section, you will learn how to fix your IP address so that it does not change between reboots. The steps are as follows:

- Log in to your Raspberry Pi 5 via Putty
- Check whether DHCP is active on your Raspberry Pi (it should normally be active):

```
pi@raspberrypi:~ $ sudo service dhcpcd status
```

- If DHCP is not active, activate it by entering the following commands:

```
pi@raspberrypi:~ $ sudo service dhcpcd start
```

```
pi@raspberrypi:~ $ sudo systemctl enable dhcpcd
```

- Find the IP address currently allocated to you by entering the command **ifconfig** or **hostname -I**. In this example, the IP address was: 192.168.1.251. We can use this IP address as our fixed address since no other device on the network is currently using it.

```
pi@raspberrypi:~ $ hostname -I  
192.168.1.251 2a00:23c7:868d:7b01:1562:5802:73c0:1ff6  
pi@raspberrypi:~ $
```

- Find the IP address of your router by entering the command **ip r**. In this example, the IP address was: 192.168.1.254

```
pi@raspberrypi:~ $ ip r  
default via 192.168.1.254 dev wlan0 proto dhcp src 192.168.1.251 metric 600  
192.168.1.0/24 dev wlan0 proto kernel scope link src 192.168.1.251 metric 600  
pi@raspberrypi:~ $
```

- Find the IP address of your DNS by entering the following command. This is usually the same as your router address:

```
pi@raspberrypi:~ $ grep "nameserver" /etc/resolv.conf
```

```
pi@raspberrypi:~ $ grep "nameserver" /etc/resolv.conf
nameserver 192.168.1.254
nameserver fe80::4e1b:86ff:feb5:ba79%wlan0
pi@raspberrypi:~ $
```

- Edit file **/etc/dhcpd.conf** by entering the command:

```
pi@raspberrypi:~ $ nano /etc/dhcpd.conf
```

- Add the following lines to the bottom of the file (these will be different for your router). If these lines already exist, remove the comment character '#' at the beginning of the lines and change the lines as follows (you may notice that eth0 for Ethernet is listed):

```
interface wlan0
static_routers=192.168.1.254
static domain_name_servers=192.168.1.254
static ip_address=192.168.1.251/24
```

- Save the file by entering **CTRL + X** followed by **Y** and reboot your Raspberry Pi.
- In this example, the Raspberry Pi should reboot with the static IP address: 192.168.1.251

Creating and Running a Python Program

You will be programming your Raspberry Pi using the Python programming language. It is worthwhile to look at the creation and running of a simple Python program on your Raspberry Pi computer. In this section, the message **Hello From Raspberry Pi** will be displayed on your PC screen. There are three methods that you can create and run Python programs on your Raspberry Pi

Method 1 – Interactively from command prompt in console mode

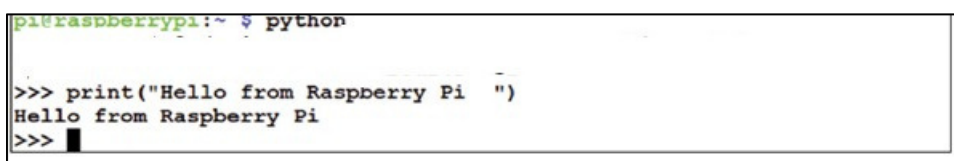
In this method, you will log in to your Raspberry Pi using the SSH and then create and run the Python program interactively. This method is excellent for small programs. The

steps are as follows:

- Login to the Raspberry Pi 5 using SSH
- At the command prompt, enter **python**. You should see the Python command mode, which is identified by three characters ">>>"
- Type the program:

```
print("Hello From Raspberry Pi ")
```

- The text will be displayed interactively on the screen, as shown in the Figure.



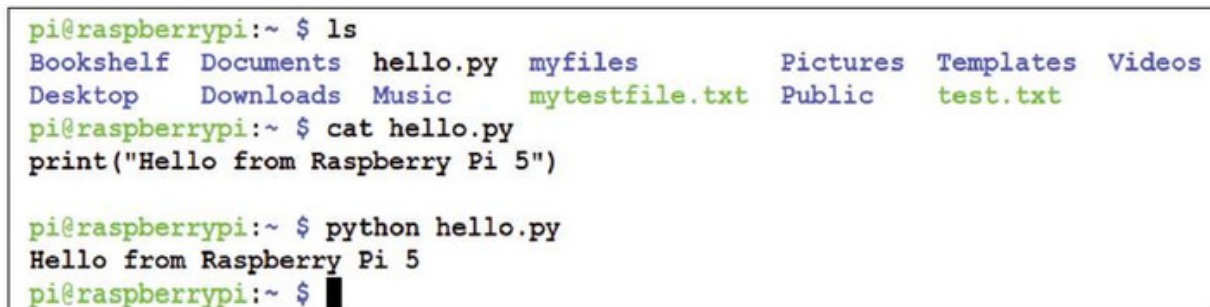
```
pi@raspberrypi:~ $ python
>>> print("Hello from Raspberry Pi ")
Hello from Raspberry Pi
>>>
```

Type Ctrl+z to exit the program

Method 2 – Create a Python file in console mode

In this method, you will log in to your Raspberry Pi using the SSH as before and then create a Python file. A Python file is simply a text file with the extension **.py**. You can use a text editor, e.g. the **nano** text editor, to create your file. In this example, a file called **hello.py** is created using the **nano** text editor. Figure shows the contents of file **hello.py**. This figure also shows how to run the file under Python. Notice that the program is run by entering the command:

```
python hello.py
```



```
pi@raspberrypi:~ $ ls
Bookshelf  Documents  hello.py  myfiles      Pictures  Templates  Videos
Desktop    Downloads  Music     mytestfile.txt  Public    test.txt

pi@raspberrypi:~ $ cat hello.py
print("Hello from Raspberry Pi 5")

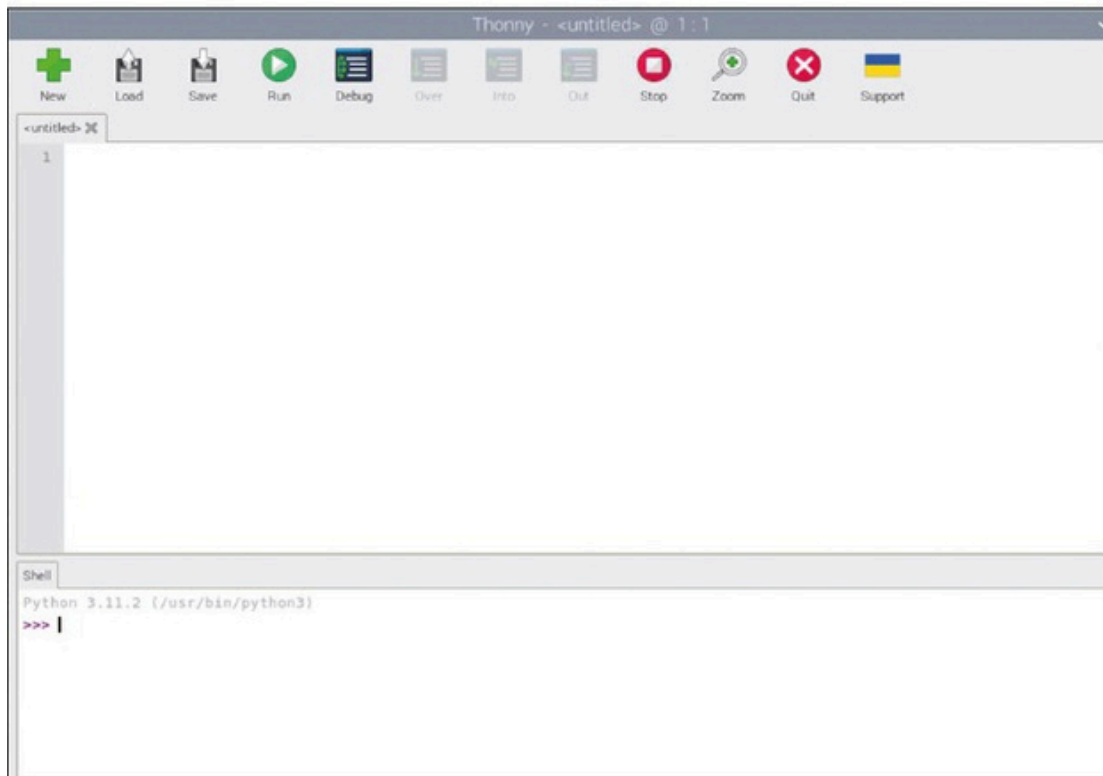
pi@raspberrypi:~ $ python hello.py
Hello from Raspberry Pi 5
pi@raspberrypi:~ $
```

Method 3 – Create a Python file in Desktop GUI mode

In this method, you can log in to your Raspberry Pi 5 using either directly connected terminal through the mini-HDMI port, or if you don't have a monitor, you can log in to the

Desktop using the VNC as described earlier and then creating and run your Python programs in GUI mode using the Thonny IDE. It is worthwhile at this stage to learn the basics of using the Thonny IDE.

- Start the Thonny IDE from the Desktop under the Programming menu. Figure shows the Thonny startup menu



The screen consists of two parts: the upper part is where you write your programs. The lower part is the shell, where small interactive programs can be written. This part is mainly used for testing code snippets.

In the upper part contain the following menu items:

- ✓ **New**: click to create a new program
- ✓ **Load**: load an existing program from a folder on Raspberry Pi
- ✓ **Save**: save an existing program on the screen to a file
- ✓ **Run**: run the program on the screen
- ✓ **Debug**: debug the program on the screen
- ✓ **Over**: used by the debugger
- ✓ **Into**: used by the debugger
- ✓ **Out**: used by the debugger

- ✓ **Stop:** stop a running program
- ✓ **Zoom:** zoom the screen
- ✓ **Quit:** Exit the Thonny IDE

The Thonny IDE must be configured before it is used to write and upload programs to your Raspberry Pi.

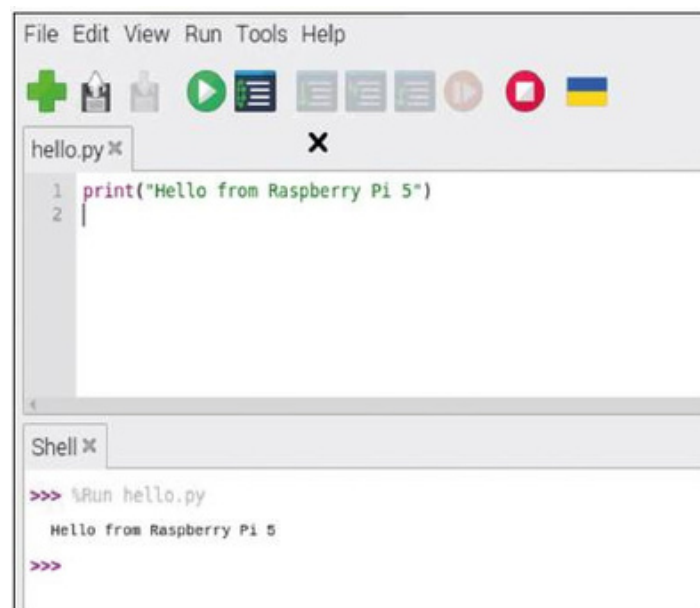
- Click the bottom-right corner of the screen to select your processor type and select Local Python 3. You are now ready to write your program. The steps are:
- Type the following code in the upper part of the screen:

```
print("Hello from Raspberry Pi 5")
```

- Click **File** → Save and save with the name hello.py



- Click the **Run** icon (green menu button at the top) to run the program. The output of the program will be displayed at the bottom of the screen as shown in Figure



You can run small programs in interactive mode by entering them at the lower part of the screen under **shell**. The results will be displayed under **shell** immediately.

Interfacing LED with Raspberry

Now, we will learn to interface the hardware component with Raspberry Pi. The component used will be LED. The Raspberry Pi 5 is connected to external electronic circuits and devices using its GPIO (General Purpose Input Output) port connector. This is a 2.54 mm, 40-pin expansion header, arranged in a 2 × 20 strip as shown in Figure. The I/O ports are numbered as

PIN	NAME		NAME	PIN
01	3.3V DC Power	Red	5V DC Power	02
03	GPIO02 (SDA1, I ² C)	Blue	5V DC Power	04
05	GPIO03 (SDL1, I ² C)	Blue	Ground	06
07	GPIO04 (GPCLK0)	Green	GPIO14 (TXD0, UART)	08
09	Ground	Black	GPIO15 (RXD0, UART)	10
11	GPIO17	Green	GPIO18 (PWM0)	12
13	GPIO27	Green	Ground	14
15	GPIO22	Green	GPIO23	16
17	3.3V DC Power	Red	GPIO24	18
19	GPIO10 (SP10_MOSI)	Purple	Ground	20
21	GPIO09 (SP10_MISO)	Purple	GPIO25	22
23	GPIO11 (SP10_CLK)	Purple	GPIO08 (SPI0_CE0_N)	24
25	Ground	Black	GPIO07 (SPI0_CE1_N)	26
27	GPIO00 (SDA0, I ² C)	Yellow	GPIO01 (SCL0, I ² C)	28
29	GPIO05	Green	Ground	30
31	GPIO06	Green	GPIO12 (PWM0)	32
33	GPIO13 (PWM1)	Green	Ground	34
35	GPIO19	Green	GPIO16	36
37	GPIO26	Green	GPIO20	38
39	Ground	Black	GPIO21	40

When the GPIO connector is at the far side of the board, the pins starting from the left of the connectors are numbered 1, 3, 5, 7, and so on, while the ones at the top are numbered as 2, 4, 6, 8 and so on. The GPIO provides 26 general-purpose bidirectional I/O pins. Some of the pins have multiple functions. For example, pins 3 and 5 are the GPIO2 and GPIO3 input-output pins, respectively. These pins can also be used as the I2C bus SDA and SCL pins, respectively. Similarly, pins 9, 10, 11 and 19 can either be used as general-purpose input-output pins, or as the SPI bus pins. Pins 8 and 10 are reserved for UART serial communication. Two power outputs are provided: +3.3 V and

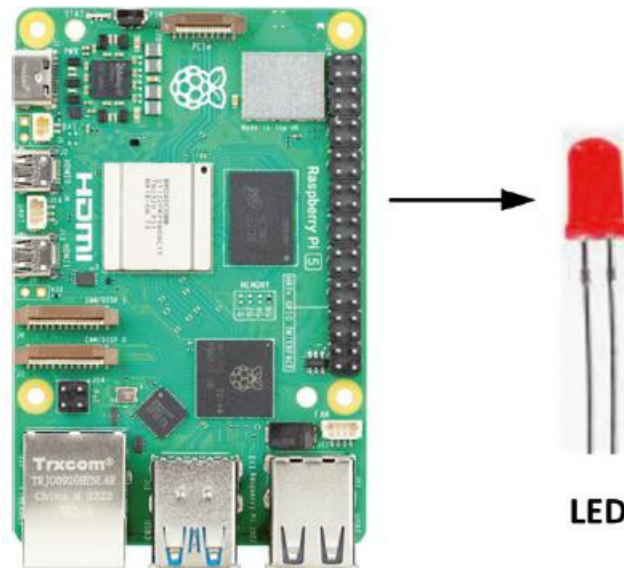
+5.0 V. The GPIO pins operate at +3.3 V logic levels (not like many other computer circuits that operate with +5 V). A pin can either be an input or an output. When configured as an output, the pin voltage is either 0 V (logic 0) or +3.3 V (logic 1). Raspberry Pi 5 is normally operated using an external power supply (e.g. a mains adapter) with +5 V output. A 3.3 V output pin can supply up to 16 mA of current.

The total current drawn from all output pins should not exceed the 51 mA limit. Care should be taken when connecting external devices to the GPIO pins, as drawing excessive currents or short-circuiting a pin can easily damage your Raspberry Pi. The amount of current that can be supplied by the 5 V pin depends on many factors, such as the current required by the Pi itself, current taken by the USB peripherals, camera current, micro-HDMI port current, and so on.

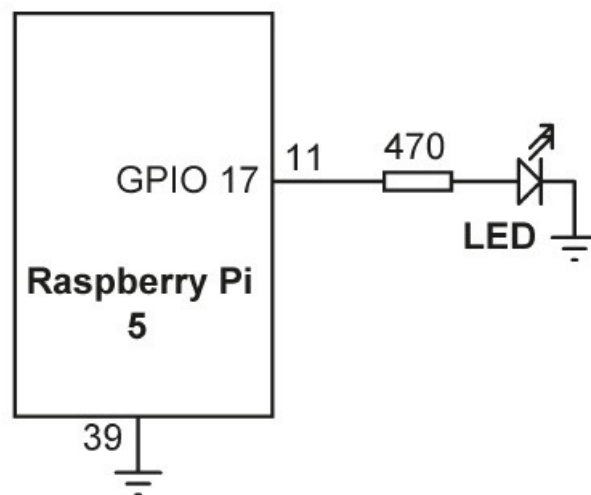
When configured as an input, a voltage above +1.7 V will be taken as logic 1, and a voltage below +1.7 V will be taken as logic 0. Care should be taken not to supply voltages greater than +3.3 V to any I/O pin, as large voltages can easily damage your Raspberry Pi. The Raspberry Pi , like others in the family, has no overvoltage protection circuitry

Flashing an LED

This is perhaps the easiest hardware project you can design using your Raspberry Pi. In this project, you will connect an LED to one of the ports of the Raspberry Pi and then flash the LED once a second. The aim of this project is to show how a simple Python program can be written and then run from a file. The project also shows how to connect an LED to a Raspberry Pi GPIO pin. In addition, the project shows how to use the GPIO library to configure and set a GPIO pin to logic 0 or 1.



Circuit diagram: The circuit diagram of the project is shown in Figure.



A small LED is connected to port pin GPIO 17 (pin 11) of the Raspberry Pi 5 through a current limiting resistor. The value of the current limiting resistor is calculated as follows: The output high voltage of a GPIO pin is 3.3 V. The voltage across an LED is approximately 1.8 V. The current through the LED depends upon the type of LED used and the amount of required brightness. Assuming that we are using a small LED, we can assume a forward LED current of about 3 mA. Then, the value of the current limiting resistor is:

$R = (3.3 - 1.8) / 0.003 = 500 \, \Omega$. We can choose a 470 Ω resistor.

In Figure above, the LED is operated in current sourcing mode where a high output from the GPIO pin drives the LED. The LED can also be operated in current sinking mode, where the other end of the LED is connected to +3.3 V supply and not to the ground. In current sinking mode, the LED is turned ON when the GPIO pin is at logic low.

Program : The program is called LED.py and the code is given below. The program can be written in Thonny. At the beginning of the program, the gpiozero and the time modules are imported to the project. The rest of the program is executed indefinitely in a **while** loop, where the LED is turned on and off with a one-second delay between each output change. Press Ctrl+C to terminate the program.

```
from gpiozero import LED # import gpiozero
from time import sleep # import time library
led = LED(17)
while True:
    led.on() # turn ON LED
    sleep(1) # wait 1 second
    led.off() # turn OFF LED
    sleep(1) # wait 1 second
```

The program is run from the console mode as follows:

```
pi@raspberrypi ~ $ python LED.py
```

If you wish to run the program from the GUI Desktop environment, you should use the VNC Viewer to get into the GUI desktop screen (unless you have a monitor connected to the Raspberry Pi 5 via a micro-HDMI cable). Then, click the Applications menu → Programming→ Thonny.

- Click File and open file LED.py, or type in the program if it is not already in your default directory.
- Click Run to run the program. You should see the LED flashing every second.
- To terminate the program, close the screen by clicking the STOP button

Lab Tasks

1. Set up Raspberry Pi card using your own memory card and set up with VNC, SSH. The board must be ready to be used via remote login.
2. Interface two LEDs with GPIOs (any two) and flash the LEDs alternately with a delay of 1 second.
3. Use Thonny to write a python code for taking inverse of a 5x5 matrix. Use Numpy.

Task:

1. Set up Raspberry Pi card using your own memory card and set up with VNC, SSH. The board must be ready to be used via remote login.

Raspberry Pi Setup using SSH and VNC:

Procedure:

- 1.Installed Raspberry Pi OS on microSD card using Raspberry Pi Imager.
- 2.Inserted the microSD card into Raspberry Pi.
- 3.Connected Raspberry Pi to Wi-Fi.
- 4.Enabled SSH using:
sudo raspi-config
- 5.Obtained IP address using:
hostname -I
- 6.Connected remotely through Putty using SSH.
- 7.Installed and configured VNC Server.
- 8.Accessed Raspberry Pi Desktop remotely using VNC Viewer.

Result:

SSH and VNC were configured successfully and Raspberry Pi was accessed remotely.

Task:

2. Interface two LEDs with GPIOs (any two) and flash the LEDs alternately with a delay of 1 second.

```
▶ from gpiozero import LED
  from time import sleep

  led1 = LED(17)
  led2 = LED(18)

  while True:

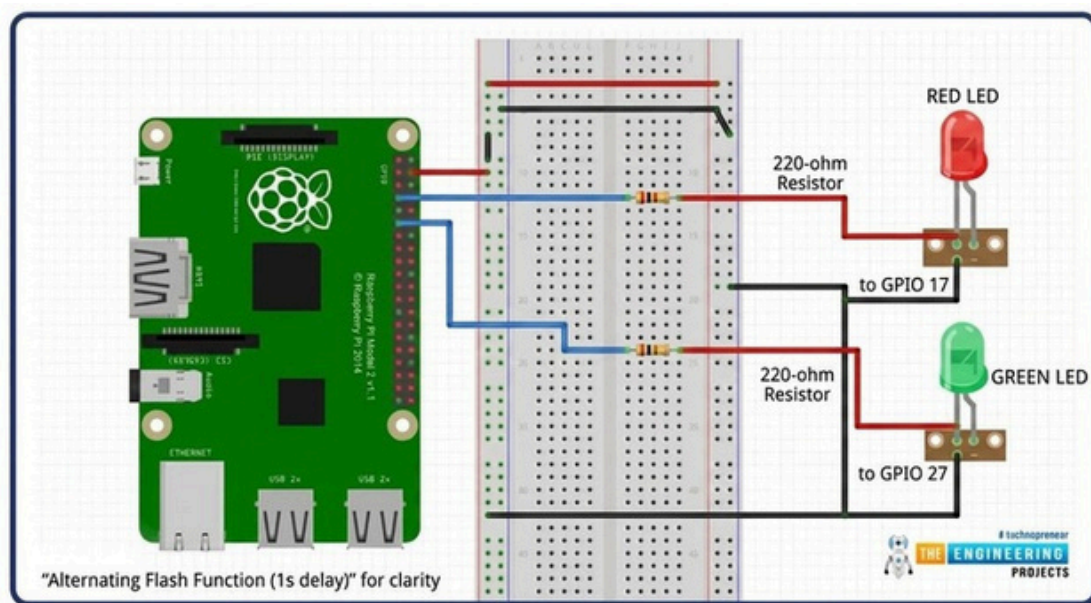
      led1.on()
      led2.off()

      sleep(1)

      led1.off()
      led2.on()

      sleep(1)
```

Block Diagram:



Expected Output

LED1 ON
LED2 OFF

(after 1 second)

LED1 OFF
LED2 ON

(after 1 second)

Task:

3. Use Thonny to write a python code for taking inverse of a 5x5 matrix. Use Numpy

Code:

```
import numpy as np

A = np.array([
    [1,2,3,4,5],
    [2,3,4,5,6],
    [1,1,1,1,1],
    [7,8,9,1,2],
    [3,4,5,6,7]
])

print("Original Matrix:")
print(A)

inverse = np.linalg.inv(A)

print("\nInverse Matrix:")
print(inverse)
```

Expected Output:

Original Matrix:

```
[[1 0 0 0 0]
 [0 1 0 0 0]
 [0 0 1 0 0]
 [0 0 0 1 0]
 [0 0 0 0 1]]
```

Inverse Matrix:

```
[[1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 1.]]
```

International Islamic University Islamabad
Faculty of Engineering and Technology
Department of Electrical Engineering

Artificial Intelligence & Machine Learning LAB

Lab No. 4

Raspberry Pi Setup for AI and Machine Learning

Name: Bushra Nazir Reg. No.: 1071-F22

S. No.	Criteria	Allocated Percentage	Level 1 (0%)	Level 2 (25%)	Level 3 (50%)	Level 4 (75%)	Level 5 (100%)	Marks Obtained
(A) <u>PSYCHOMOTOR</u> (To be judged in the lab during experiment)								
1	Practical Implementation OR Connectivity/Arrangement of Equipment	5	Absent	Degree of errors in applying procedural knowledge or Degree of errors in arrangement of equipment:				
			0	With several critical errors, Incomplete and Not Neat	With few errors, Incomplete and not Neat	With some errors, Complete but not Neat	Without errors, Complete and Neat	
				1.25	2.5	3.75	5	
2	Use of Equipment OR Use of Simulation/Programming Tool	2	Absent	Use of tools, equipment and materials with:				
			0	Limited Competence	Some Competence	Considerable Competence	Good Competence	
				0.5	1	1.5	2	
3	Safety (Optional)	0	Absent	Degree of reminders to follow safety procedure with:				
			0	Rarely Follow Safety Rules	Needs Reminders	Needs minor Reminders	Follows Safety Rules	
				0	0	0	0	
SUB-TOTAL (PT)		7	SUB-TOTAL MARKS OBTAINED (PO)					
(B) <u>COGNITIVE</u> (To be judged on the copy of experiment submitted)								
4	Data Record, Analysis and Evaluation OR Algorithm Design	1	Absent	Data record, Analysis and Evaluation or Designed Algorithm is:				
			0	Incorrect /Incomplete	Complete with some errors	Complete with few errors	Complete and Accurate	
				0.25	0.5	0.75	1	
5	Procedural Awareness (Optional)	0	Absent	Procedural awareness of the completed task is:				
			0	Inappropriate	Appropriate with some errors	Appropriate with few errors	Appropriate	
				0	0	0	0	
SUB-TOTAL (CT)		1	SUB-TOTAL MARKS OBTAINED (CO)					
(C) <u>AFFECTIVE</u> (To be judged in the lab during experiment)								
6	Level of Participation & Attitude to Achieve Individual/Group Goals	2	Absent	Degree of positive interaction as an individual or within a group:				
			0	Rare interaction	Some interaction	Acceptable interaction	Good interaction	
				0.5	1	1.5	2	
SUB-TOTAL (AT)		2	SUB-TOTAL MARKS OBTAINED (AO)					

Total Marks (PT + CT + AT) : 10

Instructor

Marks Obtained (PO + CO + AO) :